

ZigZag: Enlarging Joint Architecture-Mapping Design Space Exploration for DNN Accelerators

Linyan Mei^{ID}, Pouya Houshmand^{ID}, Vikram Jain^{ID},
Sebastian Giraldo^{ID}, and Marian Verhelst^{ID}, *Senior Member, IEEE*

Abstract—Building efficient embedded deep learning systems requires a tight co-design between DNN algorithms, hardware, and algorithm-to-hardware mapping, a.k.a. dataflow. However, owing to the large joint design space, finding an optimal solution through physical implementation becomes infeasible. To tackle this problem, several design space exploration (DSE) frameworks have emerged recently, yet they either suffer from long runtimes or a limited exploration space. This article introduces ZigZag, a rapid DSE framework for DNN accelerator architecture and mapping. ZigZag extends the common DSE with uneven mapping opportunities and smart mapping search strategies. Uneven mapping decouples operands (W/I/O), memory hierarchy, and mappings (temporal/spatial), opening up a whole new space for DSE, and thus better design points are found by ZigZag compared to other SotAs. For this, ZigZag uses an enhanced nested-for-loop format as a uniform representation to integrate algorithm, accelerator, and algorithm-to-accelerator mapping. ZigZag consists of three key components: 1) an analytical energy-performance-area Hardware Cost Estimator, 2) two Mapping Search Engines that support spatial and temporal even/uneven mapping search, and 3) an Architecture Generator that auto-explores the wide memory hierarchy design space. Benchmarking experiments against published works, in-house accelerator, and existing DSE frameworks, together with three case studies, show the reliability and capability of ZigZag. Up to 64 percent more energy-efficient solutions are found compared to other SotAs, due to ZigZag's uneven mapping capabilities.

Index Terms—DNN, accelerator, design space exploration, memory hierarchy, mapping, scheduling, dataflow, analytical model

1 INTRODUCTION

DURING the last decade, deep neural networks (DNNs) have established themselves as the principal algorithm for pattern recognition and data mining tasks, dominating the field of artificial intelligence (AI). Recent DNN models achieve greatly improved accuracies at the expense of increased depth and complexity. Executing these complex models on embedded systems is challenging due to resource and power constraints in edge devices.

To meet the constraints in these devices, a lot of research in the recent past, in both industry and academia, has been done towards developing specialized hardware accelerators [1], [2], [3], [4], [5], [6], [7] for energy-efficient and high-throughput mapping of DNN workloads. Each accelerator is designed with a different architecture (memory hierarchy and PE array) and a different choice of dataflow/mapping.

However, most of them are ad-hoc, locally optimal designs resulting from exploration of a limited design space. It is difficult to justify if the topology selected for the accelerators is the best one, given the vast design space available. Therefore, it is essential to have a framework that can rapidly explore the

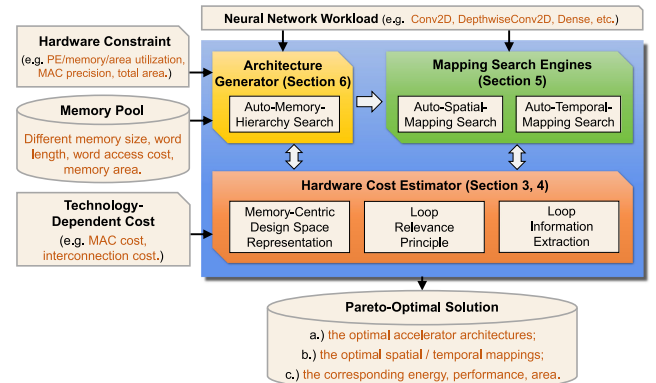


Fig. 1. ZigZag framework diagram.

available design space and guide designers to find the Pareto-optimal architectures with the optimal mappings, while taking hardware constraints, technology-dependent cost, and algorithmic workloads into account.

Many frameworks have thus emerged over the last few years targeting such hardware-software co-optimization by exploring the large design space available in the DNN accelerator system. Recent works on DSE framework in literature include Interstellar [8], [9], SMAUG [10], Accelergy [11], Dory [12], Timeloop [13], dMazeRunner [14], MAESTRO [15], and MAGnet [16].

To provide a clear view into the variety of DSE frameworks available in the SotA, Table 1 compares many SotA DSE frameworks from four perspectives. First, concerning the *hardware design space*, SotA frameworks can be

• The authors are with MICAS, The Department of Electrical Engineering (ESAT), KU Leuven, 3001 Leuven, Belgium.
E-mail: {linyan.mei, pouya.houshmand, vikram.jain, sebastian.giraldo, marian.verhelst}@esat.kuleuven.be.

Manuscript received 30 Aug. 2020; revised 19 Jan. 2021; accepted 13 Feb. 2021.
Date of publication 22 Feb. 2021; date of current version 9 July 2021.

(Corresponding author: Linyan Mei.)

Recommended for acceptance by A. Rahimi, L. Benini, S. Benatti, and T. Jang.
Digital Object Identifier no. 10.1109/TC.2021.3059962

TABLE 1
DNN Accelerator DSE Framework Comparison

Framework	Hardware Design Space	Mapping Space	Mapping Search Strategies	Cost Estimation
Timeloop+Acclergy [11], [13]	Fully flexible	Even mappings	Constraint-driven random sampling	Highly fine-grained analytical model
MAESTRO [15]	Parametrizable HW template	Even mappings	Tool-predefined dataflows	Coarse-grained analytical model
Interstellar [8]	All mem. levels shared	Even mappings	Exhaustive loop blocking/ ordering	Coarse-grained analytical model
dMazeRunner [14]	All mem. levels shared	Even mappings	Constraint-driven + data reuse pruning	Coarse-grained analytical model
MAGnet [16]	Parametrizable HW template	Even mappings	Bayesian optimiz. + random sampling	HLS hardware implementation
Dory [12]	Fixed architecture	Even mappings	Constraint programming solver [17]	Coarse-grained analytical model
SMAUG [10]	Fixed architecture	Even mappings	User-predefined fixed mapping set	Cycle-accurate SoC estimator
Ours: ZigZag	Fully flexible	Even and uneven mappings	Exhaustive/heuristic/ iterative search	Fine-grained analytical model

distinguished based on whether they support a fully flexible hardware configuration on both MAC array and memory hierarchy, like Timeloop [13]; resp. pre-define a hardware template with certain tunable parameters, like MAGnet [16]; or make other specific assumptions, such as the sharing of all memory levels for the operands W/I/O and only explore within these constraints, like Interstellar [8].

Second, in the *algorithm-to-hardware mapping space*, two sub-spaces are included, the spatial mapping space, which defines the parallelism (how neural network loop dimensions are unrolled upon the 2D MAC array); and the temporal mapping space, which defines the data fetching schedule (in which order the neural network network layer is processed). Together, they are defined as *the choice of the dataflow*. All of the SotAs only support even mappings, which usually lead to sub-optimality as will be shown in the results. Even and uneven mapping will be discussed in detail in the Section 3 of the paper.

Third, each DSE tool typically encompasses a mapping search engine, a.k.a. auto-scheduler or mapper, to find the optimal temporal/spatial mappings for mapping a certain neural network layer onto a specific accelerator architecture. Most of the DSE frameworks perform a constraint-driven search to narrow down the space and speed up the searching procedure, like dMazeRunner [14]; some formulate the scheduling process into integer constraint problems and utilize existing optimizers to solve it, like Dory [12]; the others use a partially-predefined dataflow as a hint to generate valid mappings, like MAESTRO [15]. Commonly used scheduling constraints and strategies include setting thresholds for memory/PE array utilization and data reuse factors, putting an optimization goal such as minimizing the DRAM access or the overall memory traffic, and random sampling to avoid being trapped in a local optimum.

Finally, the last column in Table 1 lists the hardware cost estimation approach adopted by each framework, in which three main categories can be identified: 1) slow but very accurate cost estimations based on High-Level Synthesis

(HLS) [16], 2) medium-speed and accurate cycle-accurate system simulators [10], and 3) fast and relatively accurate analytical models. Moreover, there are different granularity levels for analytical model [11]. Fine-grained models, like the one embedded in ZigZag, are more accurate than most of the other coarse-grained models, by distinguishing memory write cost from read cost, considering the memory word-width's impact on access energy (e.g., same-sized memories with different aspect ratio/IO bandwidth have different data access energy), and taking data pattern/stationarity into account in the memory access cost and unit MAC cost.

This work proposes ZigZag, a rapid DNN accelerator joint architecture-and-mapping DSE framework (Section 2). ZigZag mainly innovates on systematically broadening the architecture and mapping search space: enabling even/uneven auto-mapping with smart searching methods, and thus it is able to discover better design points than other frameworks. To sum up, ZigZag made the following four key contributions.

- *Memory-Centric Design Space Representation*, based on an enhanced nested-for-loop format, is proposed as a uniform representation for each design point. It opens up a whole new mapping space for the DSE by decoupling the operands (W/I/O), the memory hierarchy, and the mapping scenarios (Section 3).
- On top of this representation, a new *Loop Relevance Principle* is built, enabling the framework to extract, in a systematic and insightful way, the key information, such as memory accesses, required memory bandwidth, etc., from which the *Hardware Cost Estimator* derives the system's energy and performance values (Section 4).
- To cope with the enlarged mapping space, which contains both even and uneven mapping possibilities, the *Mapping Search Engines* are designed with *heuristics* and *iterative* search strategies to rapidly locate optimal (on energy or/and performance) spatial and temporal mapping points, supporting

mapping on a wider variety of memory hierarchy and MAC array systems (Section 5).

- ZigZag is also extended with an *Architecture Generator* to construct different DNN accelerator architectures, especially focusing on auto-generating all valid memory hierarchies under given constraints (Section 6).

Framework validation (Section 7) and three case studies (Section 8) from different design perspectives are conducted to assess the accuracy of the Hardware Cost Estimator, to show the strength of the Mapping Search Engines, and to gain insight in the vast embedded DNN system design space.

2 ZIGZAG FRAMEWORK OVERVIEW

ZigZag consists of three key components, as shown in Fig. 1, 1) an analytical energy-performance-area Hardware Cost Estimator, 2) two Mapping Search Engines that support spatial and temporal even/uneven mapping on a wide range of architectures, and 3) an Architecture Generator that auto-explores the wide memory hierarchy design space and generates all accelerator architectures (memory hierarchy plus MAC array) that meets certain constraints. These three components cooperate in synergy to enable the exploration of a much broader design space compared to other SotAs.

ZigZag's DSE flow consists of four steps: 1) The Architecture Generator takes in workloads, hardware constraints, and a user-defined memory pool to generate all valid accelerator architectures and feed the architecture information into the Hardware Cost Estimator and Mapping Search Engines; 2) The Hardware Cost Estimator computes required area; the Mapping Search Engines search for the optimal spatial and temporal mapping candidates; 3) for each architecture, the Hardware Cost Estimator evaluates all the mapping candidates found by the Mapping Search Engines in terms of energy and/or performance (i.e., latency/throughput/MAC array utilization); 4) finally, by comparing each architecture's best mappings' energy/performance/area, the optimal architectures together with their optimal mappings are found.

Besides working in this complete-DSE mode, in which the design space of both architecture and mapping are explored, ZigZag can also work in several partial-DSE modes, in which architecture and/or mapping can be partially or fully pre-defined and only open the rest of the design space for ZigZag to explore.

3 DESIGN SPACE REPRESENTATION

An enhanced data representation format is the foundation for systematically exploring the enlarged design space in ZigZag. The proposed Memory-Centric Design Space Representation

integrates three information aspects: 1) the DNN algorithm, 2) the hardware architecture (memory hierarchy and MAC array), and 3) the algorithm-to-hardware mappings (spatial and temporal). It especially excels in capturing all forms of memory sharing, loop blocking, loop ordering, and spatial loop unrolling, for each operand (W/I/O) individually and at each memory level.

Fig. 3 illustrates this proposed representation, as well as its ability to represent balanced/unbalanced memory hierarchy, and even/uneven mapping schemes. The same loop notation as in Fig. 2 is used. To explain this in depth, let's start with the first design point of Fig. 3.

In Fig. 3a, the representation defines the dataflow information of the three operands separately, one per column, using three sets of nested-for loops. Inside of each set, the *architectural levels* are represented from bottom to top (divided by the horizontal bold lines), starting from MAC level, over Register File and Global Buffer levels, all the way up to DRAM level. This *architectural level* concept will be intensively used in the next section. The close vertical bold lines indicate the boundary between different physical memory modules, as appeared in Figs. 3b and 3d. Levels not separated with a close vertical bold line share a certain memory for the corresponding operands, e.g., the DRAM level in all (a-d). For every operand, each alphanumeric pair indicates a for-loop, e.g., the first term "K 4" is equivalent to "for K = 0 to 4-1", which together with all the other for-loops constitute the *temporal mapping*. Assigning these for-loops into different architectural levels is called loop blocking; swapping the order of all the for-loops inside one level is called loop ordering. The "u" suffix after a loop name indicates parallel-for-loop or loop spatial unrolling, such as "FYu". The format "Au | Bu" is inherited from [8], meaning that both the A and B loop dimensions are spatially unrolled, which constitute the *spatial mapping*.

Comparing the four design points in Fig. 3 reveals the power of this representation: 1) it enables operands to have a different number of memory levels each, e.g., in design points (b)(d), weights have two memory levels while Input and Output have three; 2) it allows operands that have same number of memory levels to share or not-share physical memory, e.g., the Register File of Input/Output are shared in (a)(c) and are separated in (b)(d); 3) it allows operands that share one physical memory have the same (even) or different (uneven) loop blocking, e.g., all memory levels in (c) hold different loop sets for each operand.

Towards later automated mapping optimizations, it should be noted that a valid mapping needs to comply with several rules: 1) temporal loops of all operands should follow the same order to maintain the functional

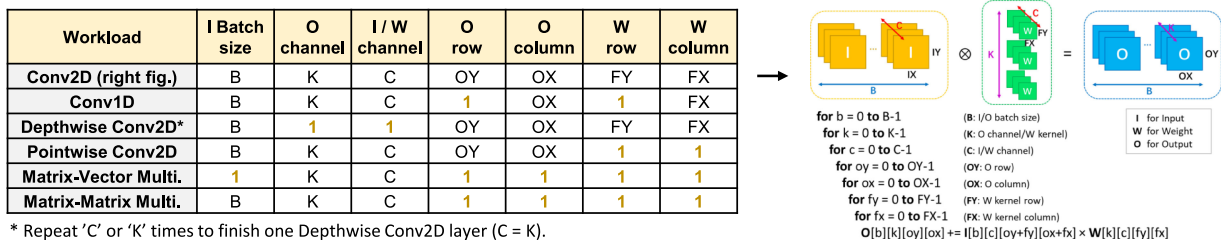


Fig. 2. Workload summary of common neural network layers. Loop notation (B, K, C, OY, OX, FY, FX) will be used in the rest of the article. Authorized licensed use limited to: UNIST. Downloaded on April 01, 2026 at 01:06:24 UTC from IEEE Xplore. Restrictions apply.

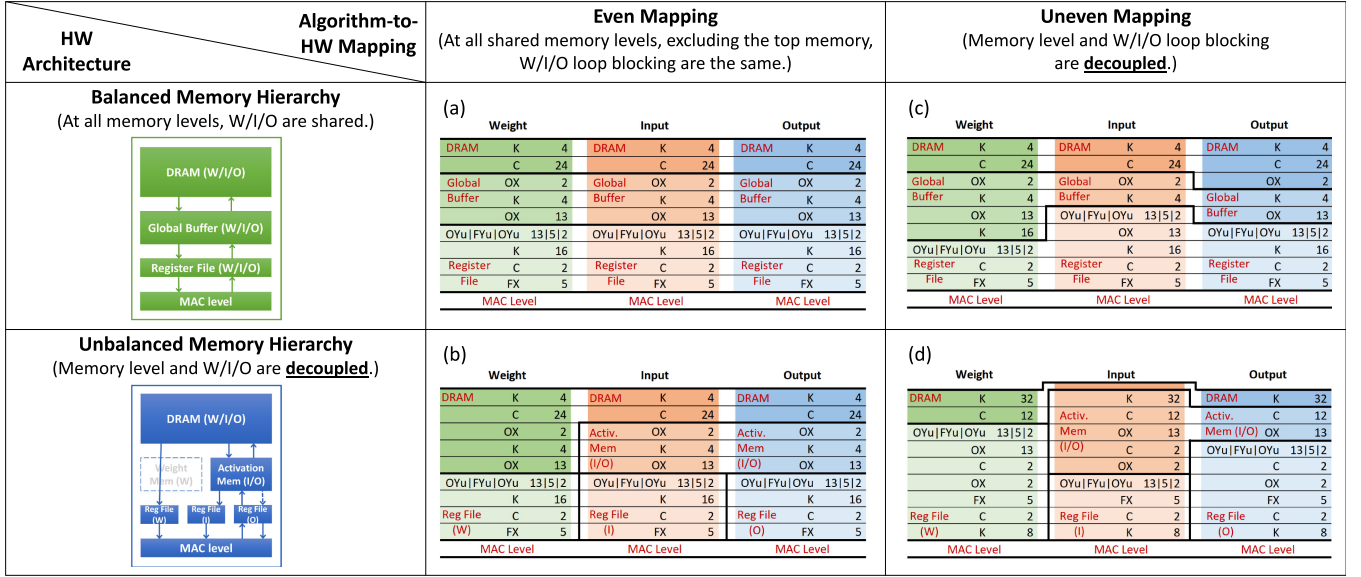


Fig. 3. Using the memory-centric design space representation to distinguish between balanced and unbalanced memory hierarchy, even and uneven mapping. The memory unrolling is not depicted in the left two memory hierarchy sketches for clarity.

equivalence; 2) together, the spatial loops of all operands should have the same unrolled loop types and dimensions (yet, there is no need for spatial loops of individual operands to follow the same order or appear at the same level) 3) spatial and temporal loops have to be assigned such that the given the memory size at each level is respected; 4) in DSE runs with a fixed hardware architecture (i.e., without architecture search), the spatial mapping needs to be consistent with the predefined memory hierarchy and Network-on-Chip (NoC).

ZigZag's loop representation is the first one that is capable of capturing all these hardware-mapping opportunities in one common structure. This paves the way to systematically and automatically extract and analyze all feasible design points, and estimate their hardware cost, as will be discussed in the next section.

4 HARDWARE COST ESTIMATOR

ZigZag Hardware Cost Estimator targets the estimation of energy, performance (PE array utilization, throughput, and latency) and area of mapping certain neural network workload onto certain accelerator architecture.

It innovates on a.) a loop relevance principle (Section 4.1), to innovate basic technology-independent hardware and data attribute in a systematic and insightful way (Section 4.2); b.) a technology- and memory-bandwidth-aware hardware cost integrator (Section 4.3), capable of extracting energy and performance.

4.1 Loop Relevance Principle

Convolutional layers are based on a 7D computing space with three 4D operands: Weight, Input, and Output; which implies not all 7 dimensions are relevant to each operand. Fig. 4 shows the loop relevance principle foundation, in which all 7 loop dimensions are categorized as relevant (r), irrelevant (ir), or partially relevant (pr) to each operand. For Weight and Output, this is straightforward since all 7 computing space dimensions are either parallel (relevant) or orthogonal (irrelevant) to their own 4D data space. Looping

through those 'r' loops indicates new data need to be fetched (for W and I) or generated (for O), while looping through those 'ir' loops indicates data reuse opportunities.

Input, however, has 'pr' loops besides the 'r' and 'ir' loops. As presented in the Conv2D example of Fig. 2, Input's dimensions IX and IY do not show up in the convolution formula directly, instead they are indirectly present through OX and FX (for IX); OY and FY (for IY). As such, OX, FX, OY, FY are denoted as partially relevant (pr) loops for Input. OX, FX (resp. OY, FY) form a 'pr' loop pair. The data reuse opportunities for Input that come from 'pr' loop pair is less straightforward and is explained as below:

The complete relation between IX and its 'pr' loop pair OX, FX is: $IX = SX \cdot (OX - 1) + SFX \cdot (FX - 1) + 1$, in which SX is the stride on input feature map IX dimension; SFX is the stride in filter FX dimension (dilated convolution). Input data reuse happens when the value of IX remains constant while the computation is looping through its data space, spatially or temporally. A simple case is that, when SX and SFX are 1, the equation becomes $IX = OX + FX - 1$, in which case, for a 'pr' loop pair, data reuse opportunities arise when the sum of their indices (OX + FX) remains constant. ZigZag can analyze various Input data reuse cases considering stride and filter dilation. For clarity, the rest of the paper will mainly focus on the simple case.

Fig. 5 provides a summary of pr-loop-pair-triggered Input data reuse. Such 'pr' creates alternative data reuse opportunities for spatially, temporally or spatio-temporally unrolled loops. For spatial unrolling, inputs can be broadcasted diagonally in a PE array, as done in Eyeriss [2], where FY and OY are spatially unrolled onto the 2D PE array, same as Fig. 5(1). It is because the sum of FY and OY remains constant along the

	B	K	C	OY	OX	FY	FX
W	×	✓	✓	×	×	✓	✓
I	✓	×	✓	?	?	?	?
O	✓	✓	×	✓	✓	×	×

✓ relevant (r)
 × irrelevant (ir)
 ? partially relevant (pr)
 ?IX/IY partially relevant to IX/IY

Fig. 4. Loop type categorized by relevance.

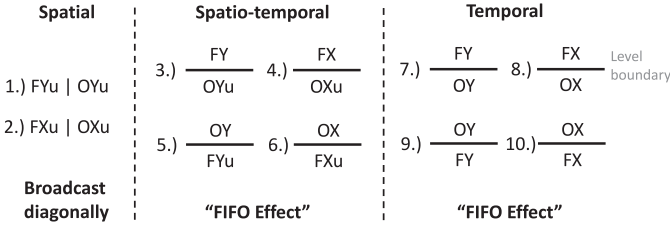


Fig. 5. pr loop patterns that trigger special Input data reuse.

diagonal direction. But note that things become more complicated when stride or dilation is not 1, in which case the broadcast angle of Inputs may no longer follow a 45 degree line. For temporal and spatio-temporal unrollings, data reuse is possible through a FIFO buffer (or a FIFO-like memory reading/writing pattern) which shifts the input data in/out over consecutive (or periodic) clock cycles. An example of this can be found in Envision [1], where OX is spatially unrolled and FX is the innermost temporal loop at the above level memory, same as Fig. 5(4), making the sum of FX and OX a constant in neighboring PE locations across consecutive cycles, enabling the reuse of Inputs in a FIFO manner.

4.2 Loop Information Extraction

The benefit of the Loop Relevance Principle is the simplification and unification of the procedure for extracting key information from the W/I/O loops sets towards estimating system energy and performance. To show the key ideas of this procedure, a summary of the major equations is provided in Table 2 and a detailed demonstration is given in Fig. 6, in which the Output loop set of Fig. 3d is analyzed (similar procedure would be repeated for Weight/Input, not shown). An in-depth discussion of each metric in Table 2 is provided below. Note that the word ‘level’ in this section refers to the *architectural level* introduced in Section 3 if not explicitly mentioned.

1.) *Data Size in individual memory unit at current level* can be derived by multiplying together the dimensionality of all the ‘r’ loops at the current level and all levels below, together with all ‘ru’ loops (spatially unrolled ‘r’ loop) at all

levels below. This can be seen in the first equation of Table 2, in which L_i means the current architectural level, $L(i-1)$ means one level below, and L_{min} means the lowest architectural level, usually the MAC level.

Let us apply this to a specific example given in Fig. 6. The required Output data storage inside the register file of each PE (16) is calculated by multiplying the dimensionality of Level-1 ‘r’ loops (the K and OX loops with resp. loop index 1 and 4); the Data Size of the Output inside of Global Buffer (5408) is calculated by multiplying the dimensionality of Level-1&2 ‘r’ loops (loop index 1, 4, 7) and Level-1 ‘ru’ loops (loop index 6.2, 6.3). Note that in the given example, no loop is assigned to the MAC level, thus we see MAC level (L_{min} or L-0 in Fig. 6) r and ir loop dimensionality both as 1. Later for other metrics’ calculation, readers can always refer to the practical case in Fig. 6 for validation.

2.) *Data Size in total at current level* can be easily calculated by multiplying the individual Data Size in each memory unit with the dimensionality of all ‘ru’ loops at current level. Notice that the unit of the Data Size is number of elements. In order to obtain number of bits, the precision of each operand needs to be considered. Generally speaking, partial outputs have a higher data precision than weights, inputs, and final outputs.

The ability to distinguish partial outputs from final outputs is critical for accurate hardware cost estimation. Zig-Zag can easily handle this through its ‘r’ versus ‘ir’ loop representation. The final output is generated at the level of the uppermost ‘ir’ loop, e.g., the ‘C 12’ loop (index 8) in Fig. 6, which in this example makes the L2 Global Buffer level the watershed for (higher precision) partial and (lower precision) final output: the output data traffic between L1 and L2 is bidirectional with partial output data precision (except for few final iterations that final outputs are generated), whereas the output data traffic between L2 and L3 is unidirectional with final output data precision.

3.) *Number of MAC Operation supported by current level data size* is calculated by multiplying together all the loops’ dimensionality (‘r’, ‘ir’, ‘ru’, and ‘iru’) from the lowest architectural level up to the current level.

TABLE 2
Equations for Loop Information Extraction

Metrics	Comment	Equation
Data Size @ Level i	Data Size in individual unit	$\prod_{L_{min}}^{L_i} r \cdot \prod_{L_{min}}^{L(i-1)} ru$
	Data Size in total	$\prod_{L_{min}}^{L_i} r \cdot \prod_{L_{min}}^{L_i} ru$
MAC Operation @ Level i	Supported by its Data Size	$\prod_{L_{min}}^{L_i} r \cdot \prod_{L_{min}}^{L_i} ru \cdot \prod_{L_{min}}^{L_i} ir \cdot \prod_{L_{min}}^{L_i} iru$
Turnaround Cycles @ Level i	Supported by its Data Size	$\prod_{L_{min}}^{L_i} r \cdot \prod_{L_{min}}^{L_i} ir$
Data Reuse Factor @ Level i	Total data reuse factor (Spatial & Temporal)	$\prod_{L_i} ir \cdot \prod_{L_i} iru$
Unit Count @ Level i	Total active unit count	$\prod_{L_i}^{L_{max}} ru \cdot \prod_{L_i}^{L_{max}} iru$
Memory Access Count (between Level i and i+1)	Write access for W and I; read access for O	$Total\ MAC\ Operation / \prod_{L_{min}}^{L_i} Total\ Data\ Reuse\ Factor$
Required Memory Bandwidth (between Level i and i+1)	With double-buffering	$\frac{Total\ Data\ Size\ @\ Level\ i}{Turnaround\ Cycles\ @\ Level\ i}$
	Without double-buffering	$\frac{Total\ Data\ Size\ @\ Level\ i}{Turnaround\ Cycles\ @\ Level\ i} \cdot \prod_{L_i} ir_{top}$

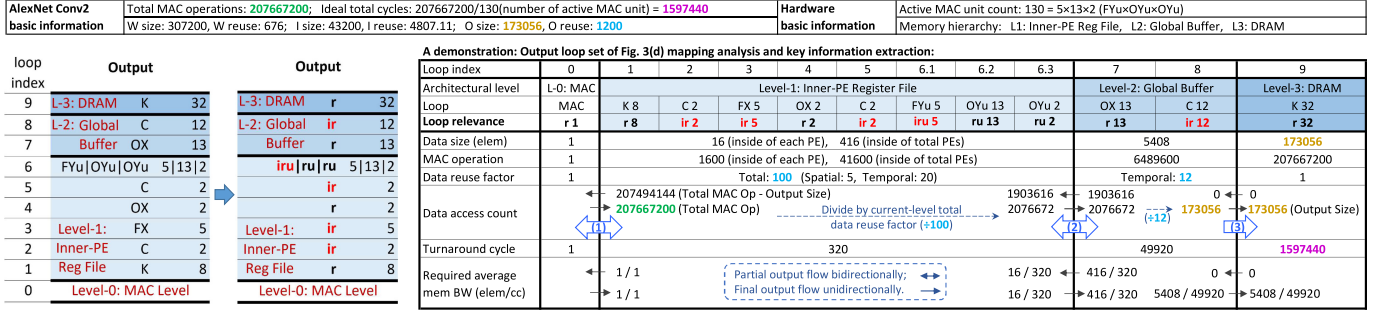


Fig. 6. A demonstration: extract loop information from Output loop set of Fig. 3d based on the loop relevance principle.

4.) *Turnaround Cycles* are the number of cycles certain memory can keep operating with the data it contains, which is an important metrics for required memory bandwidth computation. It can be calculated by multiplying together all the temporal loops' dimensionality ('r' and 'ir') from the lowest level up to the current level.

5.) *Total Data Reuse Factor* at current level is the product of all the irrelevant loops' dimensionality ('ir' and 'iru') at current level. The product of only 'ir' loops is the temporal data reuse factor, while the product of only 'iru' loops is the spatial data reuse factor.

6.) *Total Active Unit Count* is a metric that captures how many hardware components are active at certain level, which is only related to spatially unrolled loops. It can be computed as the product of all the spatial loops' dimensionality ('ru' and 'iru') from the current level up to the highest level, and it is an important metric for computing MAC array spatial utilization.

7.) *Memory Access Count*, as the core metric for later memory energy estimation, can be extracted through dividing the total MAC operation of the neural network layer by the current-and-below levels' total data reuse factor. Fig. 7 visualizes the individual loop's impact on the memory access count of the case of Fig. 3d based on this bottom-up approach. The circle markers indicate the boundary of the memory levels, showing the actual number of memory accesses at each memory level for each operand. The (1)(2)(3) points marked on Fig. 7 (both left and right) are corresponding to the (1)(2)(3) data access arrow locations in Fig. 6.

8.) *Required memory bandwidth* is the minimum bandwidth that ensures computation goes fluently without stall. It

depends on both mapping and memory settings (single-port/dual-port, with/without double buffering, etc.). Without double-buffering, writing only happens after a specific data item is fully used, resulting in a small time window. With double buffering, writing can happen all the time (in parallel with data loading), leaving a large writing time window, and thus lowering required instantaneous memory bandwidth. The bandwidth difference between these two cases is the product of all the top "ir" loop values at each memory level.

Note that due to the 'pr' loops, some changes are needed for handling Input correctly. The most important modification are the following two substitutions. One is to correctly handle data size (assuming stride is 1):

$$\prod_{L_{min}}^{L_i} r \rightarrow \prod_{L_{min}}^{L_i} r \cdot \left(\prod_{L_{min}}^{L_i} pr_1 + \prod_{L_{min}}^{L_i} pr'_1 - 1 \right) \cdot \left(\prod_{L_{min}}^{L_i} pr_2 + \prod_{L_{min}}^{L_i} pr'_2 - 1 \right),$$

in which pr_1 (pr_2) and pr'_1 (pr'_2) are a pr loop pair, like OX and FX. Another substitution is to correctly handle special Input data reuse cases like the "diagonal broadcast" and "FIFO Effect":

$$\text{Total data reuse factor @Li} \rightarrow \frac{\text{Total MAC Op@Li(+pr)}}{\text{Total Data Size@Li(+pr)}}.$$

For example, in the "FIFO Effect" setting: $\frac{FX \ 3}{OXu \ 4}$, the lower-level data reuse factor should equal to $\frac{(4 \times 3) \text{ MAC Op}}{(4+3-1) \text{ data}} = 2$ instead of $\frac{(4 \times 1) \text{ MAC Op}}{(4+1-1) \text{ data}} = 1$ by taking the "FIFO effect"-triggering 'pr' loop FX 3 into account.

4.3 Hardware Cost Integrator

The Hardware Cost Integrator aims at integrating the extracted technology-independent loop information with the technology-dependent characteristics to estimate the final hardware cost and performance.

1.) *Area*: Area estimation is straightforward by summing up all the on-chip memory and datapath area. ZigZag also provides an active/dark silicon area report based on the spatial mapping and memory unrolling.

2.) *Energy*: MAC computation energy and memory access energy are taken into account in ZigZag. MAC computation energy is divided into two parts: the active MAC energy

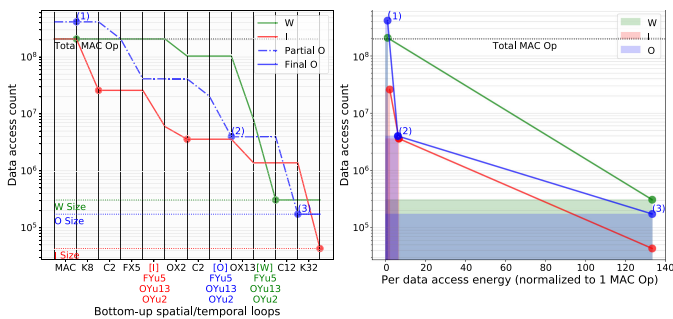


Fig. 7. (Left) Visualization of the impact of individual loops of Fig. 3d on the data access count, and (right) on the energy consumed by different memory levels. Each colored block (right) represents a memory level for certain operand; the area of the block indicates the total energy consumption at that memory level (data access count × per data access energy).

and the idle MAC energy. Both are estimated by first calculating the total active MAC count (determined by the workload) and idle MAC count (determined by the PE array's under-utilization), and subsequently multiplying the corresponding number of MAC operations with the corresponding averaged single-MAC-operation energy (active and idle).

Memory access energy is calculated by multiplying the memory access count (computed previously) with the corresponding memory per-data-access energy, taking into account the memory size, the potential memory bitwidth mismatch overhead, operand precision, and data stationarity.

3.) *Latency/Throughput*: PE array utilization, throughput, and latency are tightly related and can be deduced from each other. A PE array's under-utilization comes from two sources: spatial under-utilization and temporal under-utilization. Spatial under-utilization results from the mismatch between spatial mapping and MAC array size. Temporal under-utilization mainly come from memory bandwidth bottlenecks during computation.

ZigZag analytically estimates both types of under-utilization. Spatial PE array utilization can be computed by dividing the number of active MAC units (determined by the spatial mapping) by the total number of physical MAC units. One way to improve spatial utilization, is to maximally use the spatial dimension for all loop-iterations except for the last one, which is denoted as “*greedy mapping*”. For example, if we want to unroll a loop of dimension 20 onto an array with a size of 8×1 , the mapper based on prime factorization would run for 4 iterations of a 5-way spatial mapping, resulting in a spatial utilization of $(5 + 5 + 5 + 5)/(8 + 8 + 8 + 8) = 62.5\%$. The greedy mapper on the other hand, would propose a solution that runs for 2 iterations with 8-way spatial parallelism, and a last clock cycle that exploits the array 4-way, whose spatial utilization is $(8 + 8 + 4)/(8 + 8 + 8) = 83.3\%$. Users can enable or disable the spatial greedy mapping functionality to their needs in ZigZag.

Temporal PE array under-utilization or temporal stalls are calculated by first comparing the actual memory bandwidth with the required memory bandwidth (computed previously), which is tightly coupled to memory scenarios and mapping. The next step is analyzing ideal memory data transfer duration and data transfer period, i.e., understanding how long and how often one memory is working. Finally, stalls due to limited memory bandwidth can be calculated by the equation shown and explained in Fig. 8.

The practical situations are usually more complicated. ZigZag's performance analysis incorporates the following factors: memory bandwidth, memory sharing, memory type (single-/dual-port, wi./wo. double-buffering), memory unrolling, and partial/final output precision difference.

Besides estimating latency, another feature of ZigZag is its ability to detect the minimal required memory size at each memory level when executing a network layer (similar to the concept of register lifetime analysis in register allocation from computer architecture). The memory part that exceeds the “effective memory size” can theoretically be gated to save power. ZigZag provides the effective memory size analysis for each valid mapping point for each layer.

Total stalled [rd/wr] cycle @ certain memory level =

$$\left(\frac{\text{required BW} \times \text{ideal working duration}}{\text{actual BW}} - \text{ideal working duration} \right) \times \frac{\text{total cycle}}{\text{working period}}$$

Fig. 8. Computation stalls due to deficient memory bandwidth.

5 MAPPING SEARCH ENGINES

Mapping search engines generate valid spatial and temporal mappings to be fed into the cost estimator. Given that the mapping space is extremely large with even/uneven mapping possibilities ($\sim 1s-100s$ of millions), evaluating every single mapping point in order to locate the optimal one would be very time-consuming (note that it is just to map one neural network layer on a fixed architecture). To tackle this, ZigZag enables, besides an exhaustive search, novel searching methods to efficiently prune away sub-optimal mappings, reducing the number of mappings to be evaluated down by orders of magnitude, with no or little optimality loss across different searching methods.

5.1 Spatial Mapping Search Engine

Spatial mapping defines the flow of the operands in the spatial dimensions across the PE array. Depending on the mapping, a considerable amount of data reuse of each operand can be obtained spatially, which results in a reduced number of accesses to the levels in the memory hierarchy outside the array. In order to efficiently explore the spatial mapping space, three search methods have been developed: *exhaustive search*, *heuristic search v1* based on data reuse symmetry pruning and *heuristic search v2* based on the data reuse Pareto surface.

The exhaustive search generates all valid spatial mappings above a user-defined spatial utilization threshold, based on the layer dimensions and PE array size. It is capable of generating multiple dimensions of loop unrolling across each PE array dimension, e.g., unrolling $OY | (FY | K) 16 | (3 | 4)$ indicate unrolling OY loop 16 times on one PE array dimension and unrolling $FY \& K$ 3 and 4 times (together activate 12 PE units) on another PE array dimension.

While the exhaustive search can generate $\sim 10s$ of thousands of valid mappings, by adopting the proposed heuristic v1 and v2 methods, the spatial mappings to be evaluated can be reduced by 3–69 times depending on the layer characteristics according to our experiments, as depicted in Fig. 9. Both heuristic search v1 and v2 do not introduce any optimality loss, regarding energy or latency.

5.1.1 Heuristic Search v1 Based on Data Reuse Symmetry Pruning

Before any mapping selection, a neural network layer inherently has a specific maximum data reuse factor for each of its 3 operands, depending on the layer shape and size. A specific spatial mapping exploits, for each operand, part of these reuse opportunities in spatial dimension. The residual data reuse opportunities remain available to the temporal mapping. Yet, multiple spatial mappings can result in an equal data reuse

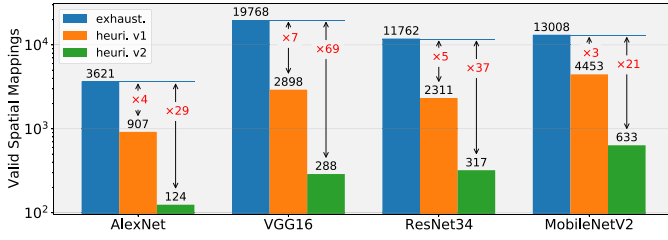


Fig. 9. Comparison between different spatial mapping search methods with 4 different neural networks. Both heuristic v1 and v2 can find the global optimal spatial mapping points as exhaustive search does.

distribution between spatial and temporal domain for each operand. As a result, their temporal mapping opportunities are identical. Such a group with equivalent, or symmetrical spatial-temporal data reuse distribution should only be considered once for further temporal mapping search.

An example is that for a neural network layer with $OX=OY$ and $FX=FY$, unrolling $OX|FX \ 7|3$ and unrolling $OY|FY \ 7|3$ are equivalent and thus one of them can be skipped for the next step temporal mapping search, while for a neural network layer with $OX \neq OY$ or $FX \neq FY$, unrolling $OX|FX \ 7|3$ and unrolling $OY|FY \ 7|3$ are no longer equivalent since they now play different roles in overall data reuse distribution, and thus should both be considered.

5.1.2 Heuristic Search v2 Based on the Data Reuse Pareto Surface

In light of the previous heuristic v1 search, in the next step the temporal mapping search engine still has to generate valid temporal mappings for each non-symmetrical spatial mapping found.

In order to further prune away sub-optimal spatial mappings, a Pareto surface of the spatial data reuse for each spatial mapping is identified in the operand space ($W/I/O$): only those spatial mappings which demonstrate Pareto-optimal data reuse along weight/input/output are processed for further evaluation.

Those mappings that are not on this Pareto surface correspond to dataflows that would require a large amount of data reuse along at least one of the operands to be handled temporally, which in turn would correspond to a larger amount of memory accesses to the upper levels in the memory hierarchy. As a consequence they can be safely ignored without losing optimality.

Given that each spatial mapping is then individually sent to the temporal mapping search engine, the speed-up in the overall search procedure is roughly proportional to the amount of spatial mappings pruned away.

5.2 Temporal Mapping Search Engine

In the temporal domain, each neural network layer is expressed as three sets of nested-for loops (for I/W/O resp.) that determine the order of execution of the MAC operations within each PE. Since the operands of the loops are distributed across the memory hierarchy, each loop is mapped to a memory level. The order, size, and layer dimension of the nested-loops determines the *temporal mapping*.

By adopting the enhanced Memory-Centric Design Space Representation and using the concept of *virtual memory level*,

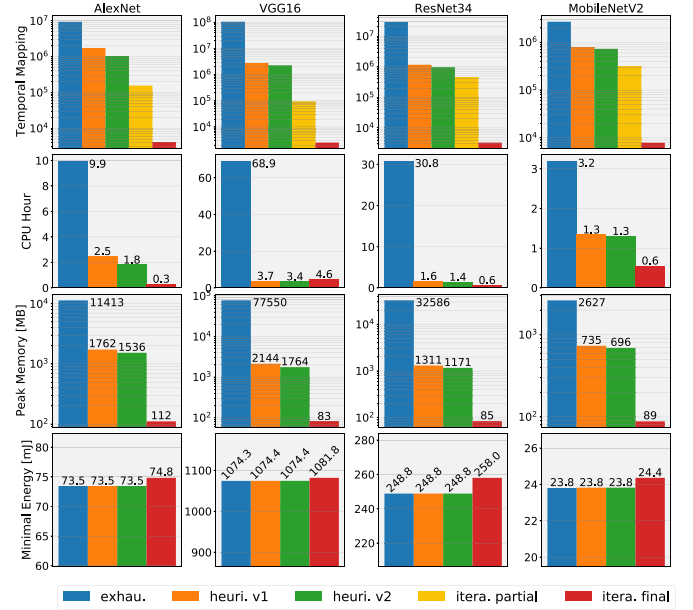


Fig. 10. Comparison between different temporal mapping search methods with 4 different neural networks on (row 1) number of temporal mappings evaluated, (row 2) elapsed time in CPU hours, (row 3) peak CPU memory usage, and (row 4) minimal mapping energy found. This experiment is carried out on an Intel Xeon Gold 6126 CPU.

the proposed temporal mapping search engine efficiently supports producing even and uneven loop schemes on balanced and unbalanced memory hierarchies that present shared and/or non-shared memory levels between different operands. Note that ZigZag has significantly enlarged the mapping design space compared to all the previous works, which enables users to obtain more optimal design points, as will be later shown in Section 7.

With this enlarged mapping space, however, it is important to efficiently prune away suboptimal mappings. Smart search methods outlined in this section will show the evaluation of several orders of magnitude less mappings with respect to exhaustive search, as shown in Fig. 10, with a maximum 5 percent loss in optimality.

5.2.1 Exhaustive Search

The exhaustive search consists of two steps, shown in Fig. 11 (left): 1) the *loop blocking* step, in which all valid loop combinations are assigned to the memory levels and 2) the *loop ordering* step, in which all valid permutations of the loop assignments are generated. In order to explore all possible schedules, we treat *loop prime factors* (LPF) as the smallest sizes in which a loop can be split. These LPFs correspond to the result of the factorization of each layer dimensions (i.e., B, C, K, ...) and are the basic blocks for the search algorithm.

The loop blocking step consists of exhaustively exploring how temporal loops can be assigned in the memory hierarchy. At each virtual memory level, the core procedure includes three steps: 1) generate all possible LPF combinations from the LPF's still to be assigned, 2) compute for each operand which of these combinations fit in the memory level and 3) generate a new set of partial mappings starting from each fitting combination found. The memory level at which the LPF combinations are allocated is initially set as the innermost one in the hierarchy for each operand; after

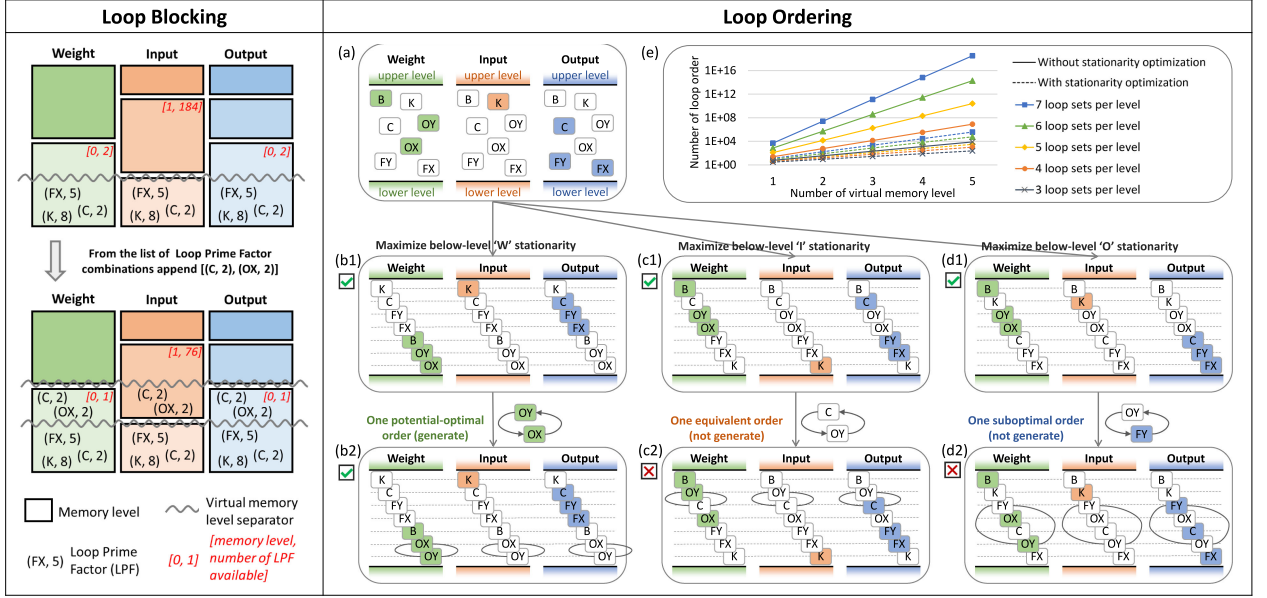


Fig. 11. Loop blocking assignment step (left); loop ordering based on data stationarity maximization (right). With the increasing of virtual memory levels and the number of nested loops at each virtual memory level, loop ordering possibility increases exponentially. By applying data stationarity optimization, the reduction on loop orders is significant. (c2) and (c1) orderings are equivalent for Input since C and OY both are relevant loops that do not change the Input's stationarity at the below memory level that comes from K. (d2) is sub-optimal compared to (d1) since for Output, OY is a relevant loop and FX is an irrelevant loop, swapping them breaks the Output's stationarity loop chain formed by FX-FY-C.

each assignment operation, the memory level to which the combinations will be assigned in the next iteration is updated. The update procedure checks if, considering the remaining LPFs to be assigned, the current memory level can still fit any additional assignment; if not, then the memory level considered for assignment in the next iteration will become the next memory level in the hierarchy.

After the loop assignment, the set of valid mapping schemes is fed into the loop ordering step. In this step, within each virtual memory level, all permutations of the loops are considered, and each becomes a different valid temporal mapping. The number of mapping schemes generated in this way grows exponentially with the number of LPFs and the number of virtual memory levels. To reduce this mapping explosion, smarter search methods are also provided.

5.2.2 Heuristic Search v1 and v2

In order to prune away sub-optimal mappings before the cost evaluation stage, two heuristic principles are applied: *data stationarity maximization* (heuristic search v1) and *data reuse pruning* (heuristic search v2).

Heuristic search v1 is applied at the loop ordering stage, as depicted in Fig. 11 Loop Ordering and Fig. 12 column 3: for each virtual memory level, instead of doing complete loop permutations, only those permutations that maximize data stationarity for each operand at the below memory level are generated, which, in the loop format, is equivalent to putting all the 'ir' loops of each operand close to the lower level boundary. By doing so, the number of valid temporal mappings to be evaluated drops by an order of magnitude, as shown in Fig. 10, without introducing optimality loss.

On top of heuristic search v1, heuristic search v2 is further applied after the LPF assignment and before the loop ordering optimization, as depicted in Fig. 12 column 4: the data reuse factor for each operand at each level in the memory hierarchy

can be extracted at this point since the loop types and their size are known (even though their loop ordering at each level is unknown). If a particular combination of LPFs causes the data-reuse factor to be 1 at a specific level for one/more operand(s) (excluding the top memory level, e.g., DRAM), it indicates at that memory level every data element of that operand stored is accessed only once (i.e., no data reuse), and therefore having or not having that memory level does not reduce the number of accesses to higher memory levels in the hierarchy, which makes it a suboptimal scenario.

It is important to note that this rule does not hold for Input data, as even for certain mapping scheme with data reuse equal to 1 at a memory level, this level may exhibit the FIFO effect opportunity (cross-level data reuse), explained in Section 4.1, and this mapping may still be an optimal mapping.

This second pruning step further reduces the number of temporal mappings to be evaluated by up to 40 percent as seen in the results reported in Fig. 10, without introducing optimality loss. However, sometimes we do observe that data reuse pruning method introduces sub-optimality when the memory hierarchy is over-dimensioned for mapping a tiny neural network layer or a layer with very few LPFs, such that there are always memory levels without data reuse opportunities, no matter how loop assignment is done.

5.2.3 Iterative Search Based on Early-Stage Cost Evaluation

Instead of generating an exhaustive list of loop blocking schemes first and possibly pruning away the sub-optimal ones, the last search strategy proposed explores the mapping space in an iterative way. Starting from the innermost level in the hierarchy, an iteration step consists of finding the set of LPFs (that constitute a virtual level) that causes the largest amount of energy savings. The saving value is derived by estimating the energy of the partial mapping on

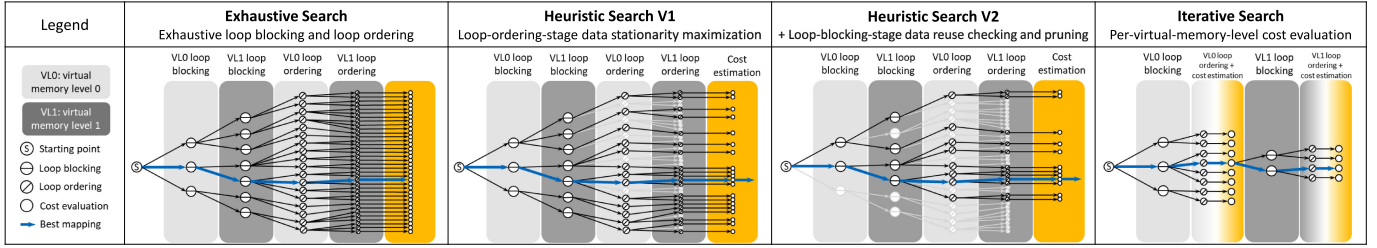


Fig. 12. Temporal mapping search engine algorithms comparison. While exhaustive search generates all valid mapping schemes ($\sim 1\text{s}$ – 100s million), heuristics are required to prune away sub-optimal mappings. Heuristic Search v1 prunes mappings at loop-ordering stage, and heuristic v2 prunes at loop blocking and loop ordering stage. Iterative search prunes at each loop assignment iteration beside applying the previous heuristics.

the whole memory hierarchy. After each iteration the best loop assignment and loop order found for current virtual memory level is stacked upon those previously found, as portrayed in Fig. 12, last column.

Iterative search requires much less CPU time/memory compared to other three search methods, as present in Fig. 10. But since this search method analyzes partial schemes and ignores the influence of the upper levels in the hierarchy when making decisions at a lower level, it might reach a sub-optimal point (within 5 percent more energy). Given the pros and cons, iterative search is still an attractive tradeoff option for many DSEs.

6 ARCHITECTURE GENERATOR

While the two mapping search engines are able to efficiently scan the dataflow space for optimal points, the architecture on which the mapping to be applied may not be the optimal one. For example, having bigger register files within each PE might have positive consequences due to higher data reuse closer to the MAC level, but it might limit the size of the buffer levels out of the array because of the limited area available on-chip and therefore limit the reduction of off-chip accesses. As a consequence, depending on the neural network workload, the memory size, the cost of data access, the word length per access, etc., an optimal point exists in the architecture space as well.

In order to also explore different hardware architectures, Architecture Generator is able to exhaustively generate all architectures that fit within a user-defined area constraint as well as other user-defined design guidelines, such as to have at least a size ratio of 8 between consecutive memory levels. To carry out this task, it first draws from a memory pool different combinations (with considering memory unrolling), and if the combination fits within the area constraints and meets the design guidelines, it then proceeds in assigning operand(s) to each memory instance (with considering different operands' memory sharing possibilities), as shown in Fig. 13.

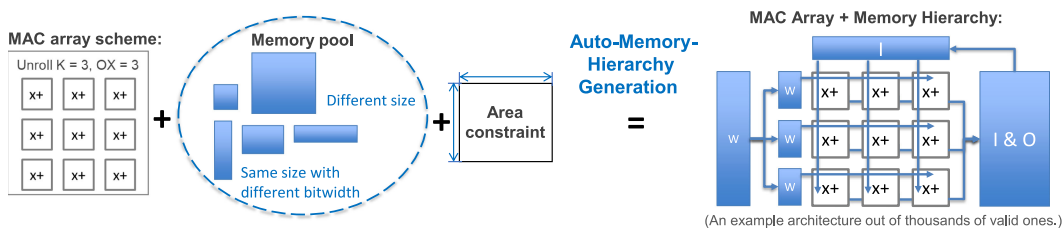


Fig. 13. Memory hierarchy generator overview.

Authorized licensed use limited to: UNIST. Downloaded on April 01, 2026 at 01:06:24 UTC from IEEE Xplore. Restrictions apply.

The output of this stage is a list of valid hardware architectures, each with a different memory hierarchy, which are sequentially fed to the mapping search engines to find the optimal spatial and temporal mappings. When all hierarchies are analyzed, the optimal memory hierarchy and its optimal mapping for energy and latency will be identified.

ZigZag allows to run the architecture exploration for a single neural network layer, a complete neural network, as well as multiple complete neural networks, as will be demonstrated in Section 8.

7 VALIDATION

The hardware cost model and the mapping search engines are validated with three methodologies: a.) against measured results of published chips; b.) against in-house post-synthesis extracted energy and performance data; c.) against other DNN accelerator DSE frameworks.

First, we model the dataflow and hardware architectures of both Eyeriss [2] and ENVISION [1] and compare the estimated energy (left bars) with their reported values (right bars), as depicted in Fig. 14. The resulting energy values, normalized with respect to the cost of a single MAC, are shown for full precision operation without voltage scaling or sparsity reduction. The estimated values are within an acceptable 5 percent, resp. 7.5 percent error margin.

Second, to also validate the throughput model, validation is performed against a complete in-house accelerator simulated post-synthesis. The results in Fig. 15 show a maximum error of 6 percent of energy and 4 percent of PE array utilization.

Finally, the validation of the cost model as well as the mapping search engines is carried out against a SotA DSE framework, Timeloop [13] + Accelergy [11].

ZigZag and Timeloop's mapping search engines are compared on Eyeriss architecture with AlexNet [18] and ResNet-34 [19] (all the unique conv. layers in ResNet-34).

The experiment is carried out in three steps. Step 1, we let Timeloop do a free search to find its optimal spatial and temporal mappings for each layer under test (Fig. 16 all the

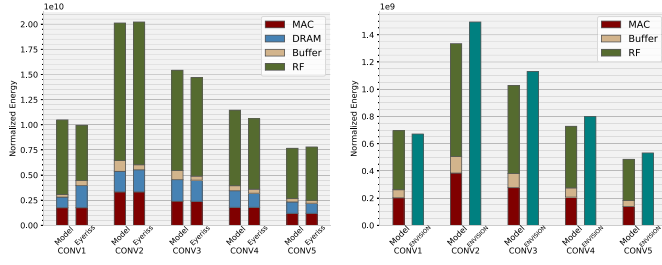


Fig. 14. Cost model validation of AlexNet [18] Conv. layers on Eyeriss [2] (left) and ENVISION [1] (right).

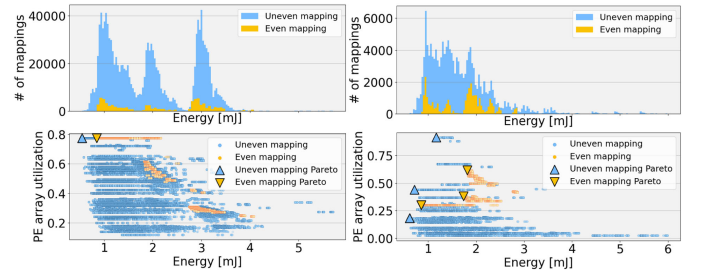


Fig. 17. Even/uneven temporal mapping's impact on energy and PE array utilization (throughput) for difference spatial unrollings: $OY_u | FY_u | Ku 13 | 5 | 2$ (left) and $OY_u | OY_u | Cu 13 | 2 | 6$ (right).

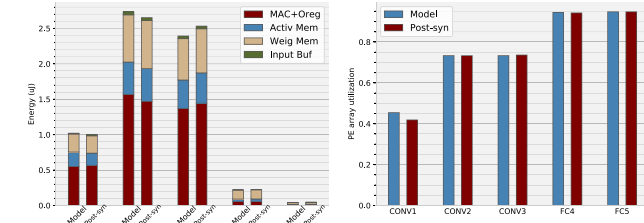


Fig. 15. Cost model validation against an in-house accelerator's post-synthesis results with a voice recognition workload on energy (left) and PE array utilization (right).

left bars in each group of three, labeled “TL”). Step 2, all the optimal mappings found by Timeloop are validated in ZigZag, demonstrating that the two cost models match well with an average error $< 5\%$ (the middle bars, labeled “TL/ZZ”). Step 3, we let ZigZag do a free search to find the optimal mappings based on the spatial/temporal heuristic search v2 methods (the right bars, labeled “ZZ”). Theoretically, there should be one last step to verify the optimal mappings found by ZigZag within the Timeloop framework, but since the mapping representation space of Timeloop does not support uneven mapping, it is not possible to carry out this reverse verification.

From the results of Fig. 16 we can draw two major conclusions: 1) ZigZag's cost model and Timeloop's cost model (Accelergy) match well (comparing the left and middle bars); 2) ZigZag's mapping search engine outperforms Timeloop's, because of the uneven mapping support (comparing the middle and right bars). Up to 64 percent memory energy savings (DRAM + Buffer + RF) can be achieved by these improved mapping schemes.

8 CASE STUDIES

To show the strength of ZigZag and use it to extract insights from the vast design space, three different case studies are

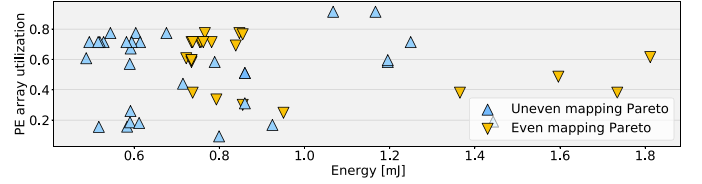


Fig. 18. The collection of Pareto-optimum temporal mappings of all 16 different spatial unrollings.

conducted. In these studies, all memory access energies are extracted from CACTI7 [20] in 65nm CMOS technology, assuming 8-bit fixed point precision for W/I/final-O, 16-bit fixed point precision for partial O.

8.1 Case Study 1: Impact of Even / Uneven Mapping

Case study 1 focuses on answering two questions: 1) how big is the impact of temporal and spatial mapping on energy and throughput, when fixing both the neural network workload and the hardware architecture (memory hierarchy and PE array size)?; 2) how large is the benefit of uneven mapping over even mapping in terms of energy and performance? In the experiment, the neural network workload and hardware architecture are fixed to AlexNet convolutional layer 2, and an Eyeriss-like architecture with a memory bandwidth assumption of 16 bit/cycle for the inner-PE register file and 64 bit/cycle for the global buffer.

This experiment consist of two steps. First, the Spatial Mapping Search Engine finds 16 spatial mappings by means of the heuristic v2 method explained in Section 5.1. Second, for each spatial mapping found, the Temporal Mapping Search Engine searches for all Pareto-optimal schedules considering energy and PE array utilization (throughput). The results are shown in Figs. 17 and 18.

In Fig. 17, two temporal mapping spaces, each with different spatial mappings, are shown. Let's first analyze the

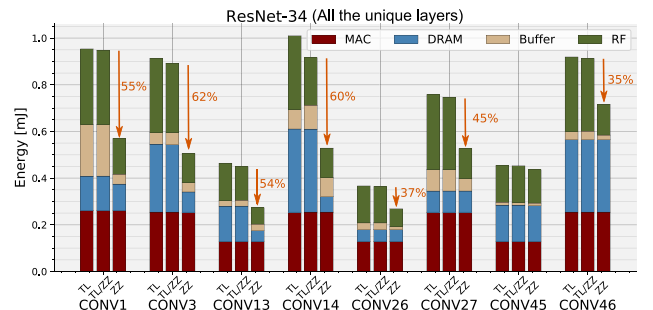
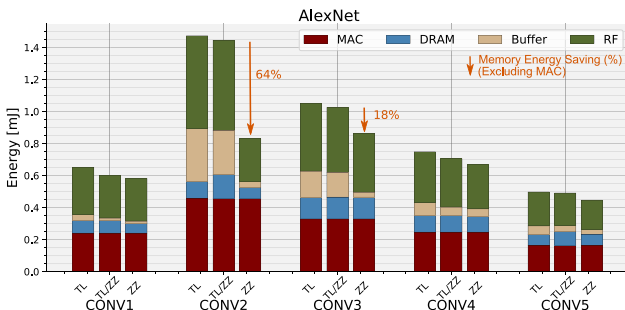


Fig. 16. Cost model and mapping search engines validation against Timeloop [13]+Accelergy [11] on AlexNet [18] (left) and ResNet34 [19] (right).
Authorized licensed use limited to: UNIST. Downloaded on April 01, 2026 at 01:06:24 UTC from IEEE Xplore. Restrictions apply.

temporal mapping's impact of even and uneven schemes separately. In the left/right figure, for even mappings, in total 73184/26944 points are found, within which up to $4.8\times/3.3\times$ energy variance and $3.2\times/2.1\times$ utilization variance are observed; for uneven mappings, in total 1296364/146940 points are found, up to $10.3\times/9.7\times$ energy variance and $7\times/42\times$ utilization variance are observed. It is clear that the temporal mapping space for uneven scheme is much larger than for even scheme. This results in significantly lower energy solutions found for uneven mappings, with an up to 30/28.6 percent lower total energy consumption compared to even mappings. In terms of PE array utilization, both the even and uneven mapping find the same optimal point in the left figure, while in the right figure, the uneven mapping achieves 27 percent higher PE array utilization, and hence 27 percent higher throughput.

Fig. 18 is the collection of the Pareto-optimal temporal mapping points (in terms of energy and utilization) for all 16 spatial unrollings. These results further confirm the ability of uneven mappings to locate better design points compared to even mappings. Here, an overall gain of up to 32.6 percent in terms of energy consumption or 12 percent in terms of throughput is achieved.

8.2 Case Study 2: Memory Hierarchy Search

Knowing that ZigZag can help designers to find the optimal spatial and temporal mapping points for a user-defined hardware architecture and neural network workload, the next challenge is determining the optimal hardware architecture given a neural network workload. This should not just be done for a single neural network layer, but for a network consisting out of multiple layers.

Case study 2 demonstrates this ability within ZigZag.

In this case study, ZigZag will search for the best memory hierarchy to process multiple neural network layers of DarkNet19 [21]. The available memory modules in the memory pool are 8 Byte, 64 Byte, 256 Byte, 8, 64, and 512 KB, in which the first 3 are a group that unrolled together with each MAC unit and the last 3 are a group without memory unrolling. ZigZag picks for each operand (W/I/O) one memory from the first memory group and pick one or zero from the second memory group. All architectures are equipped with an off-chip DRAM and a MAC array of size 12×14 .

Fig. 19 summarizes the complete results. In total, 240 different memory hierarchies are found and evaluated. All four figures share the same X-axis, which is the memory hierarchy index, from 0 to 239. The top three figures are the visualization of all memory hierarchies from each operand's perspective. The bottom figure shows the resulting total energy consumption for executing all 8 non-repetitive DarkNet19 layers on each architecture, as well as its area.

Three main observations can be summarized: 1) generally speaking, the low-energy architectures occupy large area, holding the largest on-chip memory (512 K), which largely reduces the external data transfer from DRAM; 2) when the size of the largest on-chip memory is fixed, the memory size of the inner-PE register file doesn't influence the overall energy that much, as there are clearly three energy regions one can distinguish, corresponding to an 8, 64 and 512 K upper memory; 3) the trade-off between

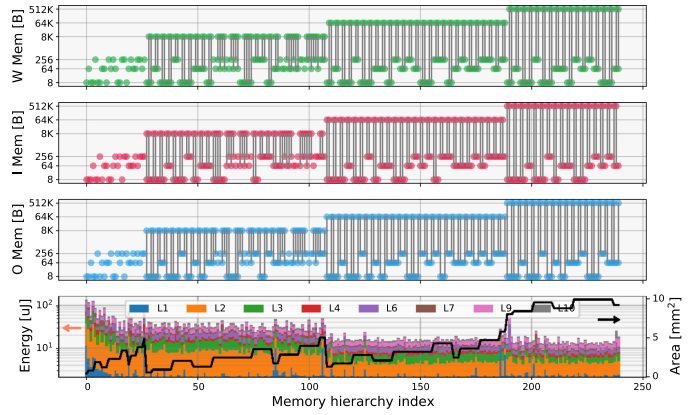


Fig. 19. Memory hierarchy search for multiple layers of DarkNet19 [21].

energy and area is quantified such that designer can make clear decisions like giving up 5 percent of energy to gain 40 percent of area saving.

8.3 Case Study 3: DNN Workload Comparison

One step beyond the single neural network (NN) study of case study 2, is the application of ZigZag to explore hardware implications across a wide variety of neural networks. Case study 3 aims to extract the optimal mappings' energy, resp. performance for 12 different NNs executing on 720 different accelerator architectures. For this, 12 popular NNs deployed on the ImageNet dataset have been selected, with their characteristics summarized in Table 4. The 720 selected hardware architectures of this study all have the same PE array size (14×16) and the same spatial unrolling ($Ox_u | Ku$ $14 | 16$), but they differ in memory hierarchy, following the configuration options listed in Table 3.

For each NN-accelerator pair, ZigZag searches the lowest-energy, resp. lowest-latency mapping. This is performed by running ZigZag temporal search engine for each layer in a particular NN and accumulating energy, resp. latency values. Thus, this results in 2 dots: 1 for the minimal latency and 1 for the minimal energy for each NN-accelerator pair, plotted on Fig. 20. Throughout the study, greedy spatial mapping is applied to all layers in order to maximize spatial PE array utilization. Heuristic search v1 is adopted for ZigZag temporal mapping search engine.

The resulting Fig. 20 reveals the global trade-off between accuracy, energy efficiency, latency, and hardware area across these 12 popular NNs, with their Pareto optimal energy and latency achievements summarized in Table 4. An analysis on Fig. 20 and Table 4 allows researchers to quickly select specific NNs based on accuracy, energy, latency or area constraints, or a combination thereof.

TABLE 3
Memory Hierarchy Options for Case Study 3

Arch. Level	Inner-PE Reg	On-chip L1	On-chip L2	Off-chip
Mem. Size Option	2 B; 32 B; 128 B	8 KB; 32 KB	0.5 MB; 2 MB	DRAM
Mem. Bandwidth	16 bit/cycle (r/w)	128 bit/cycle (read/write)		
Mem. Share Option (i.e. 1/2/3 operand(s) share same memory)	All separate	All separate; Two shared; All shared	All shared	All shared
Mem. Bypass Option	No bypass	Can bypass	Can bypass	No bypass

TABLE 4
Comparison on 12 Neural Networks' Algorithm Attribute and Hardware Performance

Neural Network	AlexNet [18]	MBV3 Small [22]	MBV1 [23]	MBV2 [24]	NASNet Small [25]	MBV3 Large [22]	ResNet 50 [19]	DenseNet 201 [26]	Xception [27]	SEResNeXt 50 [28]	IncepRes V2 [29]	NASNet Large [25]
Top-1 Accuracy (%)	56.5 (#12)	67.4 (#11)	70.6 (#10)	72 (#9)	74 (#8)	75.2 (#7)	75.3 (#6)	77.42 (#5)	79 (#4)	79.3 (#3)	80.1 (#2)	82.7 (#1)
Total MAC (GOPs)	1.07 (#7)	0.06 (#12)	0.57 (#8)	0.30 (#10)	0.56 (#9)	0.22 (#11)	3.86 (#6)	4.29 (#4)	9.48 (#3)	4.23 (#5)	13.16 (#2)	23.74 (#1)
Weight Size (MB)	24.48 (#4)	4.08 (#10)	4.01 (#11)	3.31 (#12)	5.01 (#9)	9.50 (#8)	24.32 (#6)	18.87 (#7)	24.15 (#5)	26.20 (#3)	53.15 (#2)	84.45 (#1)
Input Size (MB)	0.46 (#12)	1.90 (#11)	5.21 (#9)	6.85 (#8)	12.83 (#6)	4.92 (#10)	9.75 (#7)	23.67 (#4)	36.22 (#3)	13.71 (#5)	39.80 (#2)	137.09 (#1)
Output Size (MB)	0.63 (#12)	1.55 (#11)	4.81 (#9)	6.37 (#8)	7.57 (#6)	4.40 (#10)	10.10 (#5)	7.49 (#7)	34.17 (#2)	13.75 (#4)	23.90 (#3)	86.37 (#1)
Total Data Size (MB)	25.57 (#7)	7.53 (#12)	14.03 (#11)	16.53 (#10)	25.41 (#8)	18.82 (#9)	44.17 (#6)	50.03 (#5)	94.54 (#3)	53.66 (#4)	116.85 (#2)	307.90 (#1)
Best Energy (uJ)	20.72 (#7)	5.37 (#12)	11.03 (#11)	11.93 (#10)	19.40 (#8)	13.61 (#9)	42.05 (#6)	44.14 (#5)	90.40 (#3)	46.81 (#4)	110.30 (#2)	271.92 (#1)
Best Latency (Mcycles)	8.04 (#8)	1.75 (#12)	5.54 (#9)	4.93 (#10)	10.83 (#7)	4.63 (#11)	22.76 (#6)	23.72 (#5)	79.53 (#3)	25.59 (#4)	96.37 (#2)	209.96 (#1)

Weight/Input/Output Size is the accumulated size across all layers, assuming 8-bit precision on ImageNet data. '(#)' indicates value order, from high (#1) to low (#12), across all 12 NNs.

With NNs' accuracy, obtained Pareto-optimal solutions, and order '(#)' in Table 4, we can divide these 12 NNs into three groups: 1) NNs whose order of accuracy is smaller than the order of best energy and latency (i.e., achieve relatively high accuracy with relatively low hardware cost) are promising for embedded systems (colored green); 2) NNs whose order of accuracy is equal the order of best energy and latency (i.e., the accuracy achieved lives up to the hardware cost) are good for some applications (colored yellow); and 3) NNs whose order of accuracy is larger than the order of best energy and latency (i.e., achieve relatively low accuracy with relatively high hardware cost) are sub-optimal, and hence should be avoided (colored red).

From a HW perspective, we observe that the optimal memory hierarchy found for each NN varies, depending on the optimization goal (energy or latency), the NN's total data

size, and layer/data distribution. There are some general rules that appear. First, optimizing for latency always leads to a shallower memory hierarchy (2–3 levels) than optimizing for energy (3–4 levels). Second, the dominant operand usually prefer a shallower memory hierarchy than the non-dominant ones, e.g., for AlexNet, Weight is the dominant operand, and the two optimal memory hierarchies found (within 5 percent energy loss) are a.) 2 levels for W (128Byte, DRAM) + 3 levels for I (128Byte, 32KB, DRAM) + 3 level for O (2Byte, 32KB, DRAM), in which the 32KB is shared by I and O; and b.) 3 levels for W (128Byte, 0.5MB, DRAM) + 4 levels for I (128Byte, 32KB, 0.5MB, DRAM) + 4 level for O (2Byte, 32KB, 0.5MB, DRAM), in which the 0.5MB is shared by W, I, O. Third, different optimal memory schemes appear when optimizing for a single layer versus a complete NN. Specifically, single layer optimizations tend to favor separate memory scheme (i.e., W/I/O don't share memory levels), while memory hierarchies optimized for a complete NN seem to make extensive use of the shared memory scheme. Fourth, modern NNs, such as MobileNetV1-V3, which have a lot of group/depthwise convolution, prefer a shallow memory hierarchy for W. We observed that the previous introduced optimal memory hierarchy 'a.' for AlexNet is also among the 5 percent optimal ones for MobileNetV2, V3 small/large. A more detailed analysis on the interrelation between NN workloads and optimal HW architectures will follow in our future work.

Notice that all the results are based on a fixed PE array size with fixed spatial unrolling, CACTI7 [20]-based memory cost model, and predefined memory bandwidth. Any change on those input information could impact the final conclusion. ZigZag encourages users to plug in their own design templates and unit cost values for customized DSE.

9 CONCLUSION

This paper presented ZigZag, a DNN accelerator architecture and mapping DSE framework with enlarged mapping search space.

Three modules cooperate in synergy to enable the exploration of a much broader space of solutions with respect to other SotA DSE frameworks. First, the Architecture Generator is capable of generating all valid memory hierarchies (balanced/unbalanced, shared/separate) given a set of high-level hardware constraints. Second, the Mapping Search Engines can rapidly locate the optimal spatial and temporal mappings (even/uneven) by means of innovative search methods for

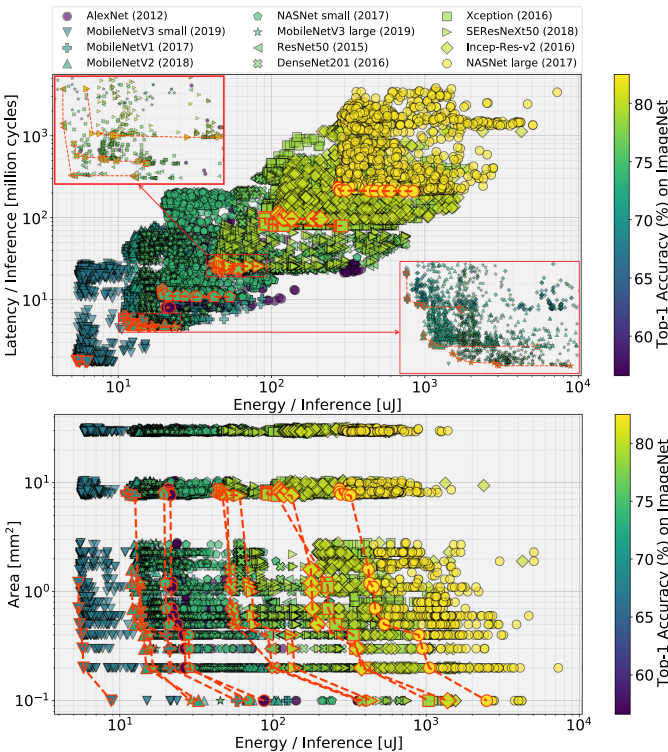


Fig. 20. Energy-latency-area comparison for mapping 12 NNs on 720 accelerator architectures each. Every NN-accelerator pair is corresponding to two points in the figure, one with min-energy mapping, one with min-latency mapping. The Pareto-optimal accelerator architectures for each NN are highlighted and connected.

any type of memory hierarchy provided by the Architecture Generator. Third, with the Memory-Centric Design Space Representation and the Loop Relevance Principle, the Hardware Cost Estimator can analytically calculate energy and latency for the mappings generated by the Mapping Search Engines. Three case studies explore the vast DNN accelerator design space from different perspectives and present the capability of ZigZag.

We are continuously improving/extending ZigZag in four major directions: 1) refined mapping search algorithms which can further reduce search time so as to handle even larger DSE experiments; 2) cross-layer mappings support to exploit scheduling across NN layers; 3) smarter hardware architecture search algorithms to also prune the hardware architecture space; 4) the extension towards emerging design concepts, such as analog-in-memory computing [30], 2.5/3D chip technology, PE array clustering, static/dynamic sparsity support, precision-scalable computing support, etc.

ZigZag is provided as an open-source tool at <https://github.com/ZigZag-Project/zigzag>. We look forward to its further exploitation in exploring the vast design space of embedded deep learning systems.

ACKNOWLEDGMENTS

This work was supported in part by the Flemish Government (AI Research Program), in part by the Fund For Scientific Research Flanders (FWO-Vlaanderen) under Grant G006718N, and in part by the EU under Grant ERC-2016-STG-715037. Linyan Mei and Pouya Houshmand contributed equally to this work.

REFERENCES

- [1] B. Moons, R. Uytterhoeven, W. Dehaene, and M. Verhelst, "Envision: A 0.26-to-10TOPS/W subword-parallel dynamic-voltage-accuracy-frequency-scalable convolutional neural network processor in 28nm FDSOI," in *Proc. IEEE Int. Solid-State Circuits Conf.*, 2017, pp. 246–247.
- [2] Y.-H. Chen, T. Krishna, J. S. Emer, and V. Sze, "Eyeriss: An energy-efficient reconfigurable accelerator for deep convolutional neural networks," *IEEE J. Solid-State Circuits*, vol. 52, no. 1, pp. 127–138, Jan. 2017.
- [3] B. Moons, D. Bankman, L. Yang, B. Murmann, and M. Verhelst, "BinarEye: An always-on energy-accuracy-scalable binary CNN processor with all memory on chip in 28 nm CMOS," in *Proc. IEEE Custom Integr. Circuits Conf.*, 2018, pp. 1–4.
- [4] D. Shin, J. Lee, J. Lee, J. Lee, and H.-J. Yoo, "DNPU: An energy-efficient deep-learning processor with heterogeneous multi-core architecture," *IEEE Micro*, vol. 38, no. 5, pp. 85–93, Sep./Oct. 2018.
- [5] Z. Du, et al., "ShiDianNao: Shifting vision processing closer to the sensor," in *Proc. 42nd Annu. Int. Symp. Comput. Archit.*, 2015, pp. 92–104.
- [6] M. Alwani, H. Chen, M. Ferdman, and P. Milder, "Fused-layer CNN accelerators," in *Proc. 49th Annu. IEEE/ACM Int. Symp. Microarchit.*, 2016, pp. 1–12.
- [7] N. P. Jouppi et al., "In-datacenter performance analysis of a tensor processing unit," in *Proc. 44th Annu. Int. Symp. Comput. Archit.*, 2017, pp. 1–12.
- [8] X. Yang et al., "Interstellar: Using Halide's scheduling language to analyze DNN accelerators," in *Proc. 25th Int. Conf. Archit. Support Program. Lang. Operating Syst.*, 2020, pp. 369–383.
- [9] X. Yang et al., "A systematic approach to blocking convolutional neural networks," 2016, *arXiv:1606.04209*.
- [10] S. L. Xi, Y. Yao, K. Bhargwaj, P. Whatmough, G.-Y. Wei, and D. Brooks, "SMAUG: End-to-end full-stack simulation infrastructure for deep learning workloads," *ACM Trans. Archit. Code Optim.*, vol. 17, 2020, Art. no. 39.

- [11] Y. N. Wu, J. S. Emer, and V. Sze, "Accelergy: An architecture-level energy estimation methodology for accelerator designs," in *Proc. IEEE/ACM Int. Conf. Comput.-Aided Des.*, 2019, pp. 1–8.
- [12] A. Burrello, F. Conti, A. Garofalo, D. Rossi, and L. Benini, "Work-in-progress: DORY: Lightweight memory hierarchy management for deep NN inference on IoT endnodes," in *Proc. Int. Conf. Hardware/Softw. Codesign Syst. Synthesis*, 2019, pp. 1–2.
- [13] A. Parashar, et al., "Timeloop: A systematic approach to DNN accelerator evaluation," in *Proc. IEEE Int. Symp. Perform. Anal. Syst. Softw.*, 2019, pp. 304–315.
- [14] S. Dave, Y. Kim, S. Avancha, K. Lee, and A. Shrivastava, "dMazeRunner: Executing perfectly nested loops on dataflow accelerators," *ACM Trans. Embedded Comput. Syst.*, vol. 18, no. 5s, 2019, Art. no. 70.
- [15] H. Kwon, P. Chatarasi, M. Pellauer, A. Parashar, V. Sarkar, and T. Krishna, "Understanding reuse, performance, and hardware cost of DNN dataflow: A data-centric approach," in *Proc. 52nd Annu. IEEE/ACM Int. Symp. Microarchit.*, 2019, pp. 754–768.
- [16] R. Venkatesan et al., "MAGNet: A modular accelerator generator for neural networks," in *Proc. IEEE/ACM Int. Conf. Comput.-Aided Des.*, 2019, pp. 1–8.
- [17] Google or-tools, 2015. [Online]. Available: <https://developers.google.com/optimization/>
- [18] A. Krizhevsky, I. Sutskever, and G. E. Hinton, "ImageNet classification with deep convolutional neural networks," in *Proc. Int. Conf. Neural Inf. Process. Syst.*, 2012, pp. 1097–1105.
- [19] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," 2015, *arXiv:1512.03385*.
- [20] R. Balasubramanian, A. B. Kahng, N. Muralimanohar, A. Shafiee, and V. Srinivas, "CACTI 7: New tools for interconnect exploration in innovative off-chip memories," *ACM Trans. Archit. Code Optim.*, vol. 14, no. 2, Jun. 2017, Art. no. 14.
- [21] J. Redmon and A. Farhadi, "YOLO9000: Better, faster, stronger," 2016, *arXiv:1612.08242*.
- [22] A. Howard, et al., "Searching for MobileNetV3," in *Proc. IEEE/CVF Int. Conf. Comput. Vis.*, 2019, pp. 1314–1324.
- [23] A. G. Howard et al., "MobileNets: Efficient convolutional neural networks for mobile vision applications," 2017, *arXiv:1704.04861*.
- [24] M. Sandler, A. Howard, M. Zhu, A. Zhmoginov, and L.-C. Chen, "MobileNetV2: Inverted residuals and linear bottlenecks," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, 2018, pp. 4510–4520.
- [25] B. Zoph, V. Vasudevan, J. Shlens, and Q. V. Le, "Learning transferable architectures for scalable image recognition," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, 2018, pp. 8697–8710.
- [26] G. Huang, Z. Liu, K. Q. Weinberger, and L. van der Maaten, "Densely connected convolutional networks," 2016, *arXiv:1608.06993*.
- [27] F. Chollet, "Xception: Deep learning with depthwise separable convolutions," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, 2017, pp. 1800–1807.
- [28] J. Hu, L. Shen, and G. Sun, "Squeeze-and-excitation networks," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, 2018, pp. 7132–7141.
- [29] C. Szegedy, S. Ioffe, V. Vanhoucke, and A. Alemi, "Inception-v4, inception-ResNet and the impact of residual connections on learning," in *Proc. AAAI Conf. Artif. Intell.*, 2017, pp. 4278–4284.
- [30] P. Houshmand et al., "Opportunities and limitations of emerging analog in-memory compute DNN architectures," in *Proc. IEEE Int. Electron Devices Meeting*, 2020, pp. 12–18.



Linyan Mei received the BSc degree in electronic science and technology from the Beijing Institute of Technology (BIT), Beijing, China, in 2016, and the MSc degree in electrical engineering from KU Leuven, Leuven, Belgium, in 2018. She is currently working toward the PhD degree in digital design for embedded machine-learning processors at the ESAT-MICAS Laboratories, KU Leuven, Leuven, Belgium. She was an Intern with imec, Leuven, Belgium, from 2017 to 2018; an Intern with Facebook, Menlo Park, California, in the first quarter of 2021. Her current research interests include energy-efficient deep neural network accelerators, algorithm-hardware co-design/optimization, and precision-scalable computing.



Pouya Houshmand received the BSc and MSc degrees in electrical engineering from the Polytechnic of Turin, Italy, in 2017 and 2019, respectively. He is currently working toward the PhD degree in architectures for deep neural network accelerators in the ESAT-MICAS Laboratories, KU Leuven, Leuven, Belgium. His current research interests include algorithm-hardware co-design, in-memory computing, and emerging technologies.



Vikram Jain received the BE degree in electrical engineering from Sathyabama University, Chennai, Tamil Nadu, India, in 2014, and the MSc degree in embedded electronics systems from the Chalmers University of Technology, Gothenburg, Sweden, in 2018. Currently, he is working toward the PhD degree in energy efficient digital acceleration and RISC-V processors for machine learning applications. In 2018, he joined ESAT-MICAS Laboratories, KU Leuven as a research assistant. His current research interests include

ML accelerators, RISC-V architecture, low power digital design, and on-chip learning.



Sebastian Giraldo received the BSc degree in electrical engineering from the Universidad de los Andes, Bogotá, Colombia, in 2012, and the MSc degree in microelectronics from the Universidade Federal do Rio Grande do Sul (UFRGS), Porto Alegre, Brazil, in 2016. Currently, he is working toward the PhD degree in digital acceleration of deep learning algorithms for voice command recognition. In 2016, he joined the ESAT-MICAS Laboratories, KU Leuven as a research assistant. His current

research interests include machine learning, speech recognition, and low-power digital design.



Marian Verhelst (Senior Member, IEEE) received the PhD degree from KU Leuven, Leuven, Belgium, in 2008. She is an associate professor with ESAT-MICAS Laboratories, KU Leuven. Her research focuses on embedded machine learning, hardware accelerators, and low-power edge processing. Before that, she worked with Intel Labs, USA, from 2008 until 2011. She is a member of the DATE and ISSCC executive committees, is TPC chair of AICAS 2020 and tinyML 2021, and TPC member of DATE and ESSCIRC. She is an SSCS

distinguished lecturer, was a member of the Young Academy of Belgium, an associate editor of the *IEEE Transactions on Very Large Scale Integration (VLSI) Systems*, *IEEE Transactions on Circuits and Systems II: Express Briefs*, and *IEEE Journal of Solid-State Circuits*. She currently holds an ERC Grant from the European Union and was the laureate of the Royal Academy of Belgium.

▷ **For more information on this or any other computing topic, please visit our Digital Library at www.computer.org/csdl.**